PRD: Nền tảng đăng ký học phần chịu tải đa trường (Dự án Công nghệ)

Oct 17, 2026 · @Nguyen Cong Phuc

Tóm tắt

Dự án xây một nền tảng đăng ký học phần chịu tải cho nhiều trường, đặt cạnh hệ thống đào tạo của từng trường, để việc đăng ký không sập và không bán vượt chỗ khi hàng chục nghìn sinh viên vào cùng lúc mở cổng.

Mỗi trường là một cấu hình riêng (giờ mở, trần tín chỉ, thứ tự ưu tiên) chạy trên cùng một nền tảng. Trọng tâm là backend event-driven phân tán, tách thành nhiều service với yêu cầu tải và độ trễ đo được.

Kết quả bàn giao: mã nguồn, bộ ADR ghi các quyết định kiến trúc, số liệu đo trước và sau mỗi cải tiến, và phân tích các sự cố gặp phải.

Bối cảnh và vấn đề

Đăng ký tín chỉ bị nghẽn hoặc sập ngay lúc mở cổng là vấn đề lặp lại ở nhiều trường đại học Việt Nam. Báo chí ghi nhận:

- Có trường mở cổng đăng ký lúc 19h và hệ thống thường sập trong 10-15 phút đầu (Tuyển Sinh Số).
- Hiệu trưởng ĐH Ngoại ngữ (ĐHQGHN) từng phải xin lỗi sinh viên vì sự cố đăng ký môn học (Tuyển Sinh Số).
- Khi hơn 35.000 sinh viên cùng đăng ký, website thường quá tải. Có sinh viên đăng ký dư 9-10 môn rồi hủy, nên một học viện đặt trần 18 tín chỉ để chặn giữ chỗ ảo (Tuyển Sinh Số).
- Có trường từng đặt phần mềm trên máy chủ của Đại học Huế và bị nghẽn khi nhiều trường cùng đăng ký trong một khoảng thời gian (Giáo dục và Thời đại).

Các trường thường dùng bộ phần mềm quản lý đào tạo trọn gói. Ví dụ ĐH Kinh tế Quốc dân dùng PSC UIS, có phân hệ đăng ký học phần trực tuyến (Tạp chí Công Thương).

Giới hạn cần nói rõ: nghẽn mạng chỉ là một phần nguyên nhân, số lớp học chưa tương xứng với nhu cầu cũng gây căng thẳng (Znews), và kỹ thuật không tạo thêm chỗ học.

Mục tiêu và không-mục-tiêu

Dự án nhắm vào độ đúng và độ ổn định của việc đăng ký lúc dồn tải, không nhắm vào việc thay hệ thống của trường.

Mục tiêu

1. Không bán vượt chỗ và giữ đúng thứ tự, kể cả khi nhiều yêu cầu đến đồng thời.
2. Chịu đỉnh tải lúc mở cổng, với độ trễ đo được (xem Yêu cầu phi chức năng).
3. Một nền tảng cho nhiều trường: mỗi trường khai báo luật riêng bằng một schema cấu hình có phiên bản.
4. Cách ly tải giữa các trường, để trường đông không làm chậm trường khác.
5. Giải thích được mọi quyết định ("vì sao bị từ chối") và điều tra được sự cố bằng trace, log, metric.

Không-mục-tiêu

- Không thay phần mềm quản lý đào tạo. Hồ sơ sinh viên, điều kiện tiên quyết và học phí vẫn năm ở hệ thống của trường.
- Không tạo thêm chỗ học và không giải bài toán xếp lớp.
- Không dùng cho đăng ký thật của trường nào trong phạm vi môn học. Dự án chạy với dữ liệu mô phỏng dựa trên quy chế công khai, và ghi rõ điều này trên giao diện và README.
- Giao diện web chỉ ở mức tối thiểu để thao tác và quan sát.

Người dùng và vai trò

Nền tảng có bốn nhóm người dùng, mỗi nhóm tương tác qua một kênh khác nhau.

Vai trò	Việc chính	Kênh
Sinh viên	Vào phòng chờ, đăng ký, xem số chỗ còn lại và vị trí hàng đợi	Web
Quản trị của trường	Khai báo cấu hình, chạy thử mô phỏng trước khi mở, xem báo cáo	Web và file cấu hình
Lập trình viên của trường	Tích hợp với hệ thống đào tạo để kiểm tra điều kiện và nhận kết quả	API và webhook
Người vận hành nền tảng	Theo dõi, điều tra sự cố, triển khai phiên bản	Dashboard, trace, log

Phạm vi MVP

MVP gồm chín chức năng, làm theo thứ tự dưới đây. Năm chức năng đầu đã đủ thành một đồ án hoàn chỉnh nếu tiến độ bị trượt.

1. Phòng chờ ảo và kiểm soát cổng vào, để dàn đều lượng người lúc mở cổng.
2. Giữ chỗ có thời hạn và xác nhận, không bán vượt chỗ.
3. Danh sách chờ, tự động đẩy người lên khi có chỗ trống.
4. Cấu hình theo trường qua schema có phiên bản: giờ mở, trần tín chỉ, nhóm ưu tiên, thời gian giữ chỗ, chính sách danh sách chờ.
5. Điểm mở rộng gọi hệ thống của trường qua API hoặc webhook để kiểm tra điều kiện đăng ký.
6. Cập nhật thời gian thực số chỗ còn lại và vị trí hàng đợi.
7. Đồng bộ kết quả về hệ thống của trường, chịu lỗi: ghi nhận trước, đồng bộ sau, thử lại.
8. Nhật ký quyết định để giải thích kết quả theo luật và phiên bản cấu hình đã áp dụng.
9. Chạy thử cấu hình bằng mô phỏng tải trước khi mở thật.

Ngoài MVP, để pha sau: triển khai riêng cho trường lớn (mô hình cell) và bảng điều khiển phân tích.

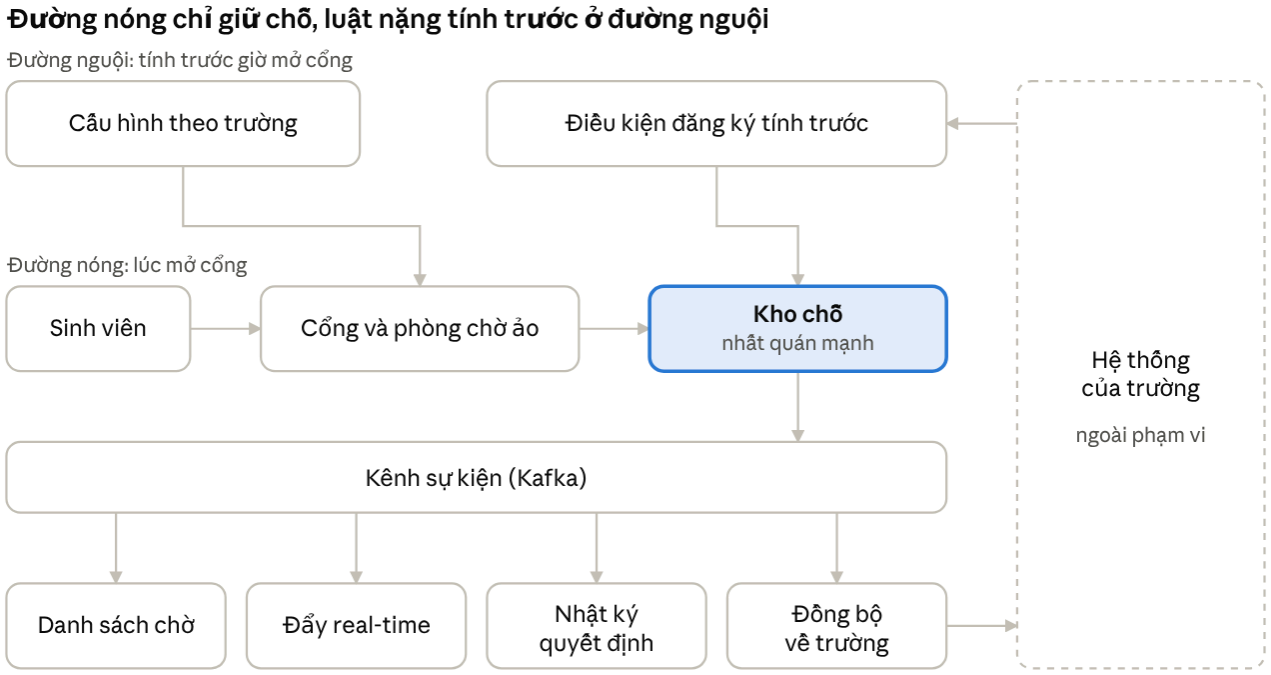
Yêu cầu phi chức năng

Các con số dưới đây là bản nháp do tác giả đặt để có thứ đo được, chưa đối chiếu với số liệu thật của trường nào. Mỗi mức sẽ được hiệu chỉnh sau khi đo, và chỉ tách thêm service khi một mức bị vi phạm.

Yêu cầu	Mức đề xuất
Đỉnh tải lúc mở cổng	20.000 sinh viên, 90% yêu cầu đến trong 60 giây đầu
Phản hồi nhận yêu cầu	p99 dưới 300 ms
Cập nhật số chỗ còn lại và vị trí hàng đợi	Dưới 2 giây
Tính đúng	0 trường hợp bán vượt chỗ; mỗi yêu cầu có đúng một kết quả cuối, kể cả khi thử lại
Chịu lỗi tích hợp	Hệ thống của trường chậm hoặc chết không làm mất đăng ký đã nhận
Cách ly giữa các trường	Một trường tăng tải không làm p99 của trường khác vượt mức trên

Yêu cầu	Mức đề xuất
Hành vi xấu	Refresh dồn dập, bot, giữ chỗ ảo bị giới hạn tốc độ; giữ chỗ tự hết hạn

Kiến trúc dự kiến



kiến trúc dự kiến · đường nóng và đường nguội

Hệ thống của trường chỉ được hỏi ở đường nguội (kiểm tra điều kiện trước giờ mở) và nhận kết quả đồng bộ sau. Lúc mở cổng, đường nóng chỉ giữ chỗ trong kho chỗ rồi phát sự kiện cho các thành phần còn lại.

Mô hình tenant giai đoạn đầu: một hệ thống dùng chung, có hạn mức và hàng đợi riêng cho từng trường, thiết kế để một trường lớn có thể chuyển sang bản triển khai riêng sau này.

Kịch bản lỗi, đánh giá và quan sát

Mỗi tuần làm một vòng: viết dự đoán trước, làm hỏng hệ thống để tái hiện, sửa bằng một pattern, viết ADR, đo trước và sau. Danh mục kịch bản ban đầu (còn mở rộng):

Kịch bản	Điều cần chứng minh	Hướng xử lý dự kiến
Hai sinh viên cùng giành chỗ cuối	Không bán vượt	Thao tác giữ chỗ nguyên tử trong kho chỗ

Kịch bản	Điều cần chứng minh	Hướng xử lý dự kiến
Sự kiện giao trùng, sinh viên bấm hai lần	Mỗi yêu cầu một kết quả	Consumer idempotent, khóa chống trùng
Service chết giữa chừng	Không mất đăng ký đã nhận	Thử lại có backoff, dead-letter, outbox
Hệ thống của trường chậm hoặc chết	Đăng ký không bị chặn	Ghi nhận trước, đồng bộ sau
Một trường tăng tải đột biến	Trường khác không bị ảnh hưởng	Hạn mức và hàng đợi riêng cho từng tenant
Refresh dồn dập, giữ chỗ ảo	Không chiếm chỗ vô hạn	Giới hạn tốc độ, giữ chỗ có hạn

Quan sát và điều tra: truyền trace context xuyên Kafka, log liên kết theo trace ID, metric về độ lệch hàng đợi và độ trễ từng bước. Câu hỏi điều tra mẫu: "vì sao yêu cầu của sinh viên A mất 4 phút, hoặc bị từ chối dù còn chỗ?". Mỗi sự cố thật gặp phải được viết thành một bản post-mortem ngắn. Phần này nối với hướng điều tra nguyên nhân gốc (RCA) trong microservices.

Lộ trình và báo cáo hăng tuần

Dự án đi theo bốn pha. Số tuần của mỗi pha sẽ chốt khi biết mốc giữa kỳ và cuối kỳ của học phần.

1. **Pha 1, nền chịu lỗi:** kho chỗ không bán vượt, idempotency, retry và dead-letter, outbox. Cổng qua pha: mỗi kịch bản lỗi tương ứng có số đo trước và sau.
2. **Pha 2, nhiều trường:** schema cấu hình có phiên bản, cách ly tenant, tách đường nóng và đường nguội. Cổng qua pha: hai trường mô phỏng với luật khác nhau chạy song song và không ảnh hưởng nhau.
3. **Pha 3, quan sát và điều tra:** trace xuyên Kafka, mô phỏng tải theo hành vi thực tế, post-mortem cho từng sự cố. Cổng qua pha: truy ra nguyên nhân của một sự cố do người khác dựng mà tác giả không biết trước.
4. **Pha 4, tổng kết:** số liệu trước và sau, báo cáo cuối, nêu hướng mở cho khóa luận.

Báo cáo hăng tuần gửi thầy, bốn dòng:

1. Đã dự đoán và làm hỏng điều gì.
2. Kết quả thực tế so với dự đoán.
3. Quyết định và đánh đổi (kèm link ADR).
4. Việc tuần sau, và phần nào tự viết, phần nào nhờ AI.

Rủi ro và giả định chưa kiểm chứng

Bốn điều dưới đây chưa được kiểm chứng, và mỗi điều có cách giảm thiểu riêng.

Rủi ro hoặc giả định	Cách giảm thiểu
Luật của các trường có thể khác nhau nhiều hơn dự kiến, làm schema cấu hình phình to	Đối chiếu quy chế đào tạo công khai của ba trường trước, chỉ đưa vào schema những điểm khác nhau thật, ghi ADR cho mỗi lần thêm
Trường có thể không cho tích hợp hoặc không có API để kiểm tra điều kiện	Thiết kế điểm tích hợp qua API và webhook chuẩn, dùng hệ thống giả lập của trường khi thử nghiệm
Tải mô phỏng do tác giả tự sinh dễ "ngoan" hơn tải thật	Mô phỏng theo hành vi đã ghi nhận (dồn lúc mở cổng, giữ chỗ rồi hủy, refresh dồn dập), nhờ người khác viết kịch bản phá hệ thống
Không có người dùng thật ở quy mô đăng ký học của trường	Chấp nhận phạm vi mô phỏng và ghi rõ trong tài liệu; nếu có điều kiện, thử ở quy mô nhỏ (suất workshop, lab, môn tự chọn nhỏ)

Nghẽn kỹ thuật chỉ là một phần nguyên nhân gây căng thẳng khi đăng ký. Dự án cải thiện độ ổn định và công bằng, không giải quyết việc thiếu chỗ học.

Câu hỏi xin ý kiến thầy

Em xin ý kiến thầy về bốn điểm sau, vì chúng quyết định phạm vi của cả dự án.

1. Phạm vi "động cơ đăng ký đứng cạnh hệ thống của trường" (không thay hệ thống đào tạo) có vừa sức và phù hợp với học phần không ạ?
2. Với phần đa trường, em nên giữ ở mức hai đến ba tenant mô phỏng dựa trên quy chế công khai, hay thầy thấy nên thu hẹp về một trường trước?
3. Thầy có hệ thống hoặc môi trường thử nào của bộ môn mà em có thể kết nối để có dữ liệu gần thực tế hơn không ạ?
4. Với mốc giữa kỳ và cuối kỳ của học phần, thầy mong em báo cáo theo mẫu bốn dòng ở trên hay theo cách khác ạ?